# IntelQcopter

## From forces to flight control

A step-by-step nonlinear model, uncertainty and measurement guide

IntelQcopter project

20 October 2026

# How to read this guide

We begin with a force in newtons, explain every symbol, build the full nonlinear rigid-body equations, then derive local transfer functions for the six existing controllers. Local models are tools for gain calculation; they do not replace the nonlinear simulation model.

A calculated value is conditional on its inputs. Our 1.2 kg flight baseline and separate 0.740 kg component estimate are both unmeasured assumptions. No number of printed decimal places turns an assumption into a measurement.

The motor KV label, propeller dimensions and ESC current rating do not identify thrust gain, delay or inertia. Unknown coefficients remain symbols or explicitly labelled scenario assumptions until measurements exist.

The current firmware keeps physical motor arming disabled. Its new battery policy warns at 35 percent estimated charge and latches disarm at 20 percent; it does not request a controlled landing. If eventually applied in air, motor thrust is removed and the aircraft falls. Storage and host builds are not physical flash-endurance or power-loss validation.

# Contents


**1 What are we measuring? 1**

1.1 Objects, symbols and units . . . 1
1.2 Mass is not weight; thrust is a force . . . 2
1.3 One vector, two coordinate descriptions . . . 2
1.4 Build the rotation explicitly . . . 3

**2 Where the translational equation comes from 4**

2.1 Start with Newton's law . . . 4
2.2 Write each differential equation without hiding $R$ . . . 4
2.3 A more general aerodynamic force model . . . 5
2.4 After commanded disarming . . . 5

**3 Why rotation needs an inertia tensor 6**

3.1 Torque comes from force and lever arm . . . 6
3.2 Build angular momentum from individual mass elements . . . 6
3.3 Derive the rotating-frame derivative . . . 7
3.4 Expand the nonlinear coupling . . . 7
3.5 Angle derivatives are not generally gyro components . . . 8

**4 The four rotors and the nonlinear actuator model 9**

4.1 Geometry and force allocation . . . 9
4.2 Retain speed-squared thrust . . . 9
4.3 Rotor angular momentum: optional additional physics . . . 10
4.4 Battery and actuator coupling . . . 10
4.5 Why a single transfer function cannot represent all of this . . . 11

**5 Equilibrium, local models and the six controllers 12**

5.1 Keep the full model as the prediction model . . . 12
5.2 Where a local linear model comes from . . . 12
5.3 The currently reproducible baseline . . . 13
5.4 Derive each local transfer function . . . 13

5.5 Why do not all controllers use nonzero P, I and D? . . . 14
5.5.1 P changes the immediate response . . . 14
5.5.2 I adds a state and removes certain steady errors . . . 14
5.5.3 D amplifies high-frequency measurement variation . . . 14
5.5.4 Why altitude does use D . . . 14
5.6 Calculate the initial six gains . . . 15
5.7 Now restore lag, drag and the complete cascade . . . 15
5.7.1 A fuller altitude coefficient match . . . 16
5.8 Sampling, limits and nonlinear assessment . . . 16

**6 Kalman estimation: uncertainty changes the gains 18**
6.1 From noisy sensors to states . . . 18
6.2 The current two-state angle/bias filter . . . 18
6.3 Derive the correction quantities . . . 19
6.4 Height and vertical velocity . . . 19
6.5 A fuller estimator is possible, but must be identified . . . 19

**7 Standard deviation is not the same as knowing the true value 20**
7.1 A fixed mass and fluctuating measurements . . . 20
7.2 Type A and Type B evaluation . . . 20
7.3 Combine uncertainty contributions . . . 21
7.3.1 Worked example: not aircraft measurements . . . 21
7.4 Correlated samples and changing conditions . . . 21

**8 Practical identification of the aircraft 22**
8.1 Records and measurement priority . . . 22
8.2 Mass and centre of mass . . . 22
8.3 Inertia from components . . . 23
8.4 Inertia from a bifilar pendulum . . . 23
8.5 Static thrust, current and RPM . . . 23
8.6 Response delay, time constant and model order . . . 24
8.7 Yaw torque . . . 24
8.8 IMU bias, noise, scale and temperature . . . 25
8.9 Height, range and camera measurements . . . 25
8.10 Voltage, current, capacity and timing . . . 25
8.11 How measurements improve gains . . . 26

**9 Calibration storage and battery behavior 27**
9.1 Persistent fits versus volatile reference data . . . 27
9.2 The journal write sequence . . . 27

9.3 Verify the gyro after boot . . . . . 27
9.4 Programming conditions and commands . . . . . 28
9.5 Requested SOC policy . . . . . 28

**10 ROS, Gazebo and implementation boundaries** **29**
10.1 Current simulation feedback path . . . . . 29
10.2 Future physical companion . . . . . 29
10.3 Build evidence and limits . . . . . 30

**References and reproducibility** **31**

# Chapter 1

# What are we measuring?

## 1.1 Objects, symbols and units

A symbol denotes a physical quantity, not an arbitrary number. If a motor produces 3 N, the number 3 depends on the chosen unit; the force itself does not. Keep units beside measurements and check every equation dimensionally.

| Symbol | Meaning | SI unit |
|---|---|---|
| $m$ | Complete aircraft mass, including battery, ESCs and attached parts | kg |
| $\boldsymbol{r} = (x, y, z)^T$ | Position of the aircraft centre of mass in world axes | m |
| $\boldsymbol{v} = \dot{\boldsymbol{r}}$ | Velocity of that centre of mass | m/s |
| $\dot{\boldsymbol{v}} = \ddot{\boldsymbol{r}}$ | Acceleration of that centre of mass | m/s$^2$ |
| $g$ | Magnitude of gravitational acceleration; baseline uses 9.80665 | m/s$^2$ |
| $T_i$ | Upward body-axis force produced by propeller $i$ | N |
| $T = \sum_i T_i$ | Sum of the four propeller thrust magnitudes | N |
| $R$ | Rotation mapping body components into world components | dimensionless |
| $\boldsymbol{\omega} = (p, q, r_b)^T$ | Body angular velocity; $r_b$ is yaw rate | rad/s |
| $J$ | Inertia tensor about the centre of mass | kg m$^2$ |
| $\boldsymbol{\tau}$ | External torque vector about the centre of mass | N m |
| $\Omega_i$ | Rotor spin speed; different from aircraft angular velocity | rad/s |
| $u_i$ | Normalized motor command, before identifying its physical response | dimensionless |

The dot is a derivative with respect to time:

$$\dot{x}(t) = \lim_{\epsilon \to 0} \frac{x(t+\epsilon) - x(t)}{\epsilon}, \tag{1.1}$$

$$\ddot{x}(t) = \frac{\mathrm{d}}{\mathrm{d}t}\dot{x}(t), \tag{1.2}$$

$$\dot{\boldsymbol{r}} = (\dot{x}, \dot{y}, \dot{z})^T. \tag{1.3}$$

For sampled data separated by $\Delta t$, a finite difference approximates the derivative. It is an approximation, not the definition:

$$\dot{x}(t_k) \approx \frac{x_k - x_{k-1}}{\Delta t}. \tag{1.4}$$

## 1.2 Mass is not weight; thrust is a force

Mass describes inertia against translational acceleration. Weight is the gravitational force. Their relationship is

$$W = mg, \qquad [W] = \mathrm{kg}\frac{\mathrm{m}}{\mathrm{s}^2} = \mathrm{N}. \tag{1.5}$$

A propeller accelerates air downward; the aerodynamic reaction exerts an upward force on the motor/airframe. That force magnitude is its thrust $T_i$. It is not a temperature, a torque, a throttle percentage or a mass.

For four parallel propeller axes, expressed in body coordinates,

$$\boldsymbol{F}_{1,B} = (0, 0, T_1)^T, \qquad \boldsymbol{F}_{2,B} = (0, 0, T_2)^T, \tag{1.6}$$

$$\boldsymbol{F}_{3,B} = (0, 0, T_3)^T, \qquad \boldsymbol{F}_{4,B} = (0, 0, T_4)^T. \tag{1.7}$$

Forces add component by component:

$$\boldsymbol{F}_{T,B} = \boldsymbol{F}_{1,B} + \boldsymbol{F}_{2,B} + \boldsymbol{F}_{3,B} + \boldsymbol{F}_{4,B} \tag{1.8}$$

$$= \begin{bmatrix} 0+0+0+0 \\ 0+0+0+0 \\ T_1+T_2+T_3+T_4 \end{bmatrix} \tag{1.9}$$

$$= \begin{bmatrix} 0 \\ 0 \\ T \end{bmatrix}, \qquad \boxed{T = T_1 + T_2 + T_3 + T_4}. \tag{1.10}$$

Only the propeller axes being parallel permits this particular scalar sum. For tilted rotor axes $\boldsymbol{n}_{i,B}$, the general expression is $\boldsymbol{F}_{T,B} = \sum_i T_i \boldsymbol{n}_{i,B}$.

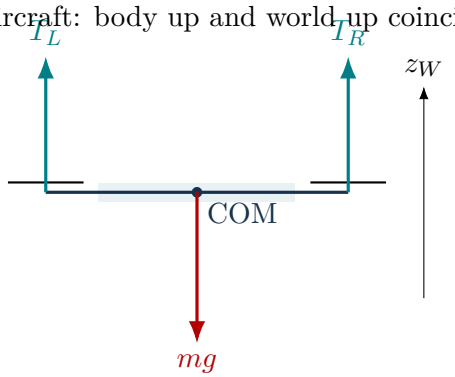



Figure 1.1: Side view groups two rotors per side. All arrows are forces; the COM is the point used for translation and torque accounting.

## 1.3 One vector, two coordinate descriptions

The world frame is fixed for a local experiment. The body frame rotates with the aircraft. Body x points forward, y left, z up. A vector must be expressed in one common frame before adding it to another vector.

Let $\boldsymbol{b}_1, \boldsymbol{b}_2, \boldsymbol{b}_3$ be the body unit axes, each expressed in world coordinates. Define

$$R = \begin{bmatrix} | & | & | \\ \boldsymbol{b}_1 & \boldsymbol{b}_2 & \boldsymbol{b}_3 \\ | & | & | \end{bmatrix}. \tag{1.11}$$

For body components $(f_x, f_y, f_z)$, the physical vector is

$$\boldsymbol{F}_W = f_x\boldsymbol{b}_1 + f_y\boldsymbol{b}_2 + f_z\boldsymbol{b}_3 \tag{1.12}$$

$$= R\begin{bmatrix} f_x \\ f_y \\ f_z \end{bmatrix}. \tag{1.13}$$

Because the axes are orthonormal,

$$R^T R = I, \qquad R^{-1} = R^T, \qquad \det R = +1. \tag{1.14}$$

Consequently $\boldsymbol{F}_B = R^T\boldsymbol{F}_W$. Multiplying by $R$ changes components, not force magnitude: $\|R\boldsymbol{F}_B\| = \|\boldsymbol{F}_B\|$.

## 1.4 Build the rotation explicitly

Use roll $\phi$, pitch $\theta$ and yaw $\psi$, with the convention $R = R_z(\psi)R_y(\theta)R_x(\phi)$. All trigonometric arguments here are radians.

$$R_x(\phi) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\phi & -\sin\phi \\ 0 & \sin\phi & \cos\phi \end{bmatrix}, \tag{1.15}$$

$$R_y(\theta) = \begin{bmatrix} \cos\theta & 0 & \sin\theta \\ 0 & 1 & 0 \\ -\sin\theta & 0 & \cos\theta \end{bmatrix}, \tag{1.16}$$

$$R_z(\psi) = \begin{bmatrix} \cos\psi & -\sin\psi & 0 \\ \sin\psi & \cos\psi & 0 \\ 0 & 0 & 1 \end{bmatrix}. \tag{1.17}$$

We only need the rotated thrust direction, so start with $\boldsymbol{e}_3 = (0, 0, 1)^T$ and perform each physical rotation:

$$R_x\boldsymbol{e}_3 = \begin{bmatrix} 0 \\ -\sin\phi \\ \cos\phi \end{bmatrix}, \tag{1.18}$$

$$R_yR_x\boldsymbol{e}_3 = \begin{bmatrix} \sin\theta\cos\phi \\ -\sin\phi \\ \cos\theta\cos\phi \end{bmatrix}, \tag{1.19}$$

$$R\boldsymbol{e}_3 = \begin{bmatrix} \cos\psi\sin\theta\cos\phi + \sin\psi\sin\phi \\ \sin\psi\sin\theta\cos\phi - \cos\psi\sin\phi \\ \cos\theta\cos\phi \end{bmatrix}. \tag{1.20}$$

The last vector is exactly the third column of $R$. Thus the world thrust is $T$ times this column. We have not used $\sin\phi \approx \phi$ or any small-angle approximation.

# Chapter 2

# Where the translational equation comes from

## 2.1 Start with Newton's law

For constant mass, linear momentum is $\boldsymbol{P} = m\boldsymbol{v}$. The momentum balance is

$$\frac{\mathrm{d}\boldsymbol{P}}{\mathrm{d}t} = \sum \boldsymbol{F}_{\text{external}}, \tag{2.1}$$

$$\frac{\mathrm{d}(m\boldsymbol{v})}{\mathrm{d}t} = \sum \boldsymbol{F}_{\text{external}}, \tag{2.2}$$

$$m\dot{\boldsymbol{v}} = \sum \boldsymbol{F}_{\text{external}}. \tag{2.3}$$

If mass changes, the second line needs more careful momentum accounting; battery discharge does not meaningfully eject aircraft mass in our model.

List every force and express it in world axes:

$$\boldsymbol{F}_{T,W} = R(0,0,T)^T \qquad \text{propellers,} \tag{2.4}$$

$$\boldsymbol{F}_{g,W} = (0,0,-mg)^T \qquad \text{gravity,} \tag{2.5}$$

$$\boldsymbol{F}_{D,W} = -d\boldsymbol{v} \qquad \text{current simple drag assumption,} \tag{2.6}$$

$$\boldsymbol{F}_{dist,W} = \text{other unmodelled external forces.} \tag{2.7}$$

Substitute these four expressions into Newton's law:

$$m\dot{\boldsymbol{v}} = \boldsymbol{F}_{T,W} + \boldsymbol{F}_{g,W} + \boldsymbol{F}_{D,W} + \boldsymbol{F}_{dist,W} \tag{2.8}$$

$$= \boxed{R\begin{bmatrix}0\\0\\T\end{bmatrix} - \begin{bmatrix}0\\0\\mg\end{bmatrix} - d\boldsymbol{v} + \boldsymbol{F}_{\text{dist}}}. \tag{2.9}$$

This is the requested equation. It is a force balance, not a guessed controller formula. Every term has units of newtons:

$$[m\dot{v}] = \mathrm{N}, \quad [RT] = \mathrm{N}, \quad [mg] = \mathrm{N}, \quad [d] = \mathrm{N\,s/m}, \quad [dv] = \mathrm{N}. \tag{2.10}$$

## 2.2 Write each differential equation without hiding $R$

Using the third-column expression derived earlier:

$$m\ddot{x} = T(\cos\psi\sin\theta\cos\phi + \sin\psi\sin\phi) - d\dot{x} + f_x, \tag{2.11}$$

$$m\ddot{y} = T(\sin\psi\sin\theta\cos\phi - \cos\psi\sin\phi) - d\dot{y} + f_y, \tag{2.12}$$
$$m\ddot{z} = T\cos\theta\cos\phi - mg - d\dot{z} + f_z. \tag{2.13}$$

These are coupled nonlinear equations because of products of changing thrust and trigonometric functions of changing attitude.

At level hover, $\phi = \theta = 0$, $\boldsymbol{v} = 0$ and disturbances are neglected:

$$0 = T - mg, \qquad T = mg, \qquad T_i = \frac{mg}{4} \quad \text{if balanced.} \tag{2.14}$$

For the assumed $m = 1.2\,\mathrm{kg}$:

$$mg = 11.76798\,\mathrm{N}, \qquad T_i = 2.941995\,\mathrm{N}. \tag{2.15}$$

At roll 30°, pitch zero and zero vertical acceleration:

$$T\cos 30^\circ = mg, \qquad T = \frac{mg}{\cos 30^\circ} \approx 13.5885\,\mathrm{N}. \tag{2.16}$$

The larger total thrust is needed because some thrust now points horizontally. Keeping $T = mg$ while tilting instead gives downward vertical acceleration.

## 2.3 A more general aerodynamic force model

The force balance remains exact as an accounting statement; a chosen drag law can still be approximate. A richer candidate is

$$\boldsymbol{v}_{\mathrm{rel}} = \boldsymbol{v}_W - \boldsymbol{w}_W, \tag{2.17}$$
$$\boldsymbol{F}_D = -D_1\boldsymbol{v}_{\mathrm{rel}} - D_2(\|\boldsymbol{v}_{\mathrm{rel}}\|\boldsymbol{v}_{\mathrm{rel}}). \tag{2.18}$$

$\boldsymbol{w}_W$ is wind velocity. $D_1, D_2$ may depend on attitude and direction. Their coefficients need data. Adding unknown coefficients without measurement does not automatically make a simulation more accurate.

## 2.4 After commanded disarming

Disarming removes commanded rotor drive; the rotor does not physically stop spinning at the exact same instant. Its motor/air dynamics determine spin-down. As $T \to 0$, neglecting drag for this illustration,

$$\ddot{z} = -g, \tag{2.19}$$
$$\dot{z}(t) = v_{z0} - gt, \tag{2.20}$$
$$z(t) = z_0 + v_{z0}t - \tfrac{1}{2}gt^2. \tag{2.21}$$

These equations describe falling, not a controlled landing. The selected 20-percent battery action requests this motor inhibition regardless of height.

# Chapter 3

# Why rotation needs an inertia tensor

## 3.1 Torque comes from force and lever arm

For a force $\boldsymbol{F}$ applied at displacement $\boldsymbol{r}_i$ from the centre of mass,

$$\boldsymbol{\tau}_i = \boldsymbol{r}_i \times \boldsymbol{F}_i. \tag{3.1}$$

For $\boldsymbol{a} = (a_x, a_y, a_z)^T$ and $\boldsymbol{b} = (b_x, b_y, b_z)^T$,

$$\boldsymbol{a} \times \boldsymbol{b} = \begin{bmatrix} a_y b_z - a_z b_y \\ a_z b_x - a_x b_z \\ a_x b_y - a_y b_x \end{bmatrix}. \tag{3.2}$$

This operation is the vector cross product. Its magnitude is $\|\boldsymbol{a}\|\|\boldsymbol{b}\| \sin\gamma$ and its direction follows the right-hand rule. It is not component-by-component multiplication.

For a rotor at $(x_i, y_i, z_i)$ with upward thrust $(0, 0, T_i)$,

$$\boldsymbol{\tau}_{T,i} = \begin{bmatrix} y_i T_i \\ -x_i T_i \\ 0 \end{bmatrix}. \tag{3.3}$$

A rotor far from the centre creates a greater roll/pitch torque for the same thrust. Its vertical height $z_i$ does not appear for this parallel upward force.

## 3.2 Build angular momentum from individual mass elements

For a mass element $\mathrm{d}m$ at body position $\boldsymbol{\rho}$, rigid-body rotation produces velocity $\boldsymbol{\omega} \times \boldsymbol{\rho}$ relative to the centre. Its angular momentum about the centre is

$$\mathrm{d}\boldsymbol{H} = \boldsymbol{\rho} \times (\boldsymbol{\omega} \times \boldsymbol{\rho})\,\mathrm{d}m \tag{3.4}$$

$$= [(\boldsymbol{\rho}^T \boldsymbol{\rho}) I - \boldsymbol{\rho}\boldsymbol{\rho}^T]\boldsymbol{\omega}\,\mathrm{d}m. \tag{3.5}$$

Integrate over the entire assembled body:

$$\boldsymbol{H}_B = J\boldsymbol{\omega}, \tag{3.6}$$

$$J = \int \left(\|\boldsymbol{\rho}\|^2 I - \boldsymbol{\rho}\boldsymbol{\rho}^T\right)\,\mathrm{d}m \tag{3.7}$$

$$= \begin{bmatrix} \int (y^2+z^2)\,\mathrm{d}m & -\int xy\,\mathrm{d}m & -\int xz\,\mathrm{d}m \\ -\int xy\,\mathrm{d}m & \int (x^2+z^2)\,\mathrm{d}m & -\int yz\,\mathrm{d}m \\ -\int xz\,\mathrm{d}m & -\int yz\,\mathrm{d}m & \int (x^2+y^2)\,\mathrm{d}m \end{bmatrix}. \tag{3.8}$$

$J_x$ therefore measures resistance to angular acceleration around body x. It depends on where mass is located, not only the total mass. Moving a battery away from an axis increases inertia about that axis.

## 3.3 Derive the rotating-frame derivative

Let a physical vector have body components $\boldsymbol{H}_B$ and world components $\boldsymbol{H}_W = R\boldsymbol{H}_B$. Differentiate with the product rule:

$$\dot{\boldsymbol{H}}_W = \dot{R}\boldsymbol{H}_B + R\dot{\boldsymbol{H}}_B. \tag{3.9}$$

Rotation kinematics gives $\dot{R} = R[\boldsymbol{\omega}]_\times$, where

$$[\boldsymbol{\omega}]_\times = \begin{bmatrix} 0 & -r_b & q \\ r_b & 0 & -p \\ -q & p & 0 \end{bmatrix}, \qquad [\boldsymbol{\omega}]_\times \boldsymbol{H}_B = \boldsymbol{\omega} \times \boldsymbol{H}_B. \tag{3.10}$$

Thus

$$\dot{\boldsymbol{H}}_W = R\left(\dot{\boldsymbol{H}}_B + \boldsymbol{\omega} \times \boldsymbol{H}_B\right). \tag{3.11}$$

Angular-momentum balance says $\dot{\boldsymbol{H}}_W = \boldsymbol{\tau}_W$. Multiply by $R^T$:

$$\boldsymbol{\tau}_B = \dot{\boldsymbol{H}}_B + \boldsymbol{\omega} \times \boldsymbol{H}_B. \tag{3.12}$$

For constant inertia, $\dot{\boldsymbol{H}}_B = J\dot{\boldsymbol{\omega}}$. Substitute $\boldsymbol{H}_B = J\boldsymbol{\omega}$:

$$\boldsymbol{\tau}_B = J\dot{\boldsymbol{\omega}} + \boldsymbol{\omega} \times (J\boldsymbol{\omega}), \tag{3.13}$$

$$J\dot{\boldsymbol{\omega}} = \boldsymbol{\tau}_B - \boldsymbol{\omega} \times (J\boldsymbol{\omega}). \tag{3.14}$$

The cross term accounts for the body axes themselves rotating while the angular momentum vector is described in those axes. It is not an extra arbitrary torque.

## 3.4 Expand the nonlinear coupling

If the selected body axes are principal axes, $J = \operatorname{diag}(J_x, J_y, J_z)$ and

$$J\boldsymbol{\omega} = (J_x p, J_y q, J_z r_b)^T, \tag{3.15}$$

$$\boldsymbol{\omega} \times (J\boldsymbol{\omega}) = \begin{bmatrix} (J_z - J_y) q r_b \\ (J_x - J_z) p r_b \\ (J_y - J_x) p q \end{bmatrix}. \tag{3.16}$$

Subtract each component and divide by its inertia:

$$\dot{p} = \frac{\tau_x + (J_y - J_z) q r_b}{J_x}, \tag{3.17}$$

$$\dot{q} = \frac{\tau_y + (J_z - J_x) r_b p}{J_y}, \tag{3.18}$$

$$\dot{r}_b = \frac{\tau_z + (J_x - J_y) p q}{J_z}. \tag{3.19}$$

For a non-diagonal measured tensor, use the full matrix inverse or solve $J\dot{\boldsymbol{\omega}} = \cdots$; do not delete products of inertia just for convenience.

## 3.5 Angle derivatives are not generally gyro components

For the stated Euler convention,

$$\begin{bmatrix}\dot{\phi}\\ \dot{\theta}\\ \dot{\psi}\end{bmatrix} = \begin{bmatrix}1 & \sin\phi\tan\theta & \cos\phi\tan\theta\\ 0 & \cos\phi & -\sin\phi\\ 0 & \sin\phi/\cos\theta & \cos\phi/\cos\theta\end{bmatrix}\begin{bmatrix}p\\ q\\ r_b\end{bmatrix}. \tag{3.20}$$

Only near modest attitudes/rates can $\dot{\phi} \approx p$ and $\dot{\theta} \approx q$ be substituted conveniently. The current independent angle/bias filters use that approximation; the physical rotational model need not.

To avoid the Euler singularity at $\cos\theta = 0$, propagate a unit quaternion. For body-to-world quaternion $q = (q_w, q_x, q_y, q_z)$,

$$\dot{q} = \frac{1}{2} q \otimes (0, p, q_{\text{rate}}, r_b), \qquad \|q\| = 1. \tag{3.21}$$

Here $q_{\text{rate}}$ means the gyro y rate, not the quaternion. A numerical implementation must keep these two meanings distinct and normalize as needed.

# Chapter 4

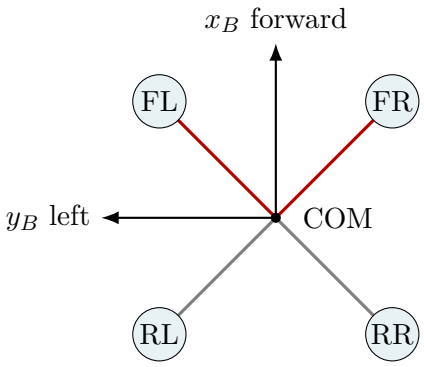
# The four rotors and the nonlinear actuator model

## 4.1 Geometry and force allocation

For motor diagonal $D$ in an X layout, $a = D/(2\sqrt{2})$:

$$(x_i, y_i) = (a, a), (a, -a), (-a, -a), (-a, a) \tag{4.1}$$

in order FL, FR, RR, RL. With reaction torque $Q_i = \kappa T_i$ and the selected yaw signs:




Top view: $z_B$ points out of the page

Figure 4.1: Logical geometry; verify actual wiring, arm orientation and propeller spin before assigning a physical motor number.

$$\begin{bmatrix} T \\ \tau_x \\ \tau_y \\ \tau_z \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ a & -a & -a & a \\ -a & -a & a & a \\ -\kappa & \kappa & -\kappa & \kappa \end{bmatrix} \begin{bmatrix} T_{FL} \\ T_{FR} \\ T_{RR} \\ T_{RL} \end{bmatrix}. \tag{4.2}$$

This allocation is a geometric consequence of $\boldsymbol{r} \times \boldsymbol{F}$ plus yaw reaction signs. A physically measured $Q_i(\Omega_i)$ can replace the constant $\kappa$ model.

## 4.2 Retain speed-squared thrust

A common aerodynamic form is

$$T_i = k_T \Omega_i^2, \qquad Q_i = k_Q \Omega_i^2. \tag{4.3}$$

More generally $k_T, k_Q$ depend on inflow, propeller condition, density and advance ratio. These functions are not identified from a 9047 label alone.

An effective motor/ESC model can be written

$$L\dot{I}_i = V_{eff,i} - R_m I_i - k_e \Omega_i, \tag{4.4}$$

$$J_r \dot{\Omega}_i = k_\tau I_i - Q_i(\Omega_i) - B_r \Omega_i, \tag{4.5}$$

$$V_{eff,i} = \mu(u_i, V_{\text{pack}}, \text{ESC settings}) V_{\text{pack}}. \tag{4.6}$$

The ESC is an electronic commutator, so these are effective averaged equations; phase/line voltage conventions and efficiency must be specified before interpreting $k_e, k_\tau$. Do not silently equate a PWM input percentage with an effective phase voltage or a thrust percentage.

The KV rating indicates an approximate no-load speed/voltage relationship. It does not supply $R_m, L, J_r, k_T, k_Q$, loaded RPM, ESC map $\mu$, or transient delay. We therefore retain these coefficients symbolically until measurement.

## 4.3 Rotor angular momentum: optional additional physics

If $\sigma_i = \pm 1$ denotes rotor spin direction,

$$\boldsymbol{H}_{\text{rot}} = \sum_i \sigma_i J_r \Omega_i \boldsymbol{e}_3. \tag{4.7}$$

For an assembled-airframe inertia $J$ defined for frozen relative rotor angles, the total rotating-body momentum balance can be expanded as

$$J\dot{\boldsymbol{\omega}} = \boldsymbol{\tau}_{\text{external}} - \boldsymbol{\omega} \times (J\boldsymbol{\omega}) - \dot{\boldsymbol{H}}_{\text{rot}} - \boldsymbol{\omega} \times \boldsymbol{H}_{\text{rot}}. \tag{4.8}$$

External rotor aerodynamic drag has sign opposite $\sigma_i$; if using the usual body reaction-torque sign $s_i$, then $s_i = -\sigma_i$. Do not count the same motor/rotor torque twice by adding it both as an external reaction and as an internal rotor acceleration contribution.

These extra terms need rotor inertia/spin information. They are model extensions, not claims that the present Gazebo adapter has identified or enabled them.

## 4.4 Battery and actuator coupling

A symbolic electrical extension is

$$V_{\text{term}} = V_{\text{oc}}(z, T_b) - I_{\text{pack}} R_0 - V_p, \tag{4.9}$$

$$\dot{V}_p = -\frac{V_p}{R_1 C_1} + \frac{I_{\text{pack}}}{C_1}, \tag{4.10}$$

$$\dot{z} = -\frac{\eta(I_{\text{pack}}, T_b) I_{\text{pack}}}{3600 C_{\text{eff}}}, \qquad z \in [0, 1]. \tag{4.11}$$

$C_{\text{eff}}$ is in Ah and positive current means discharge. Phase motor currents are not automatically identical to pack current; inverter efficiency and duty matter. Voltage sag changes available speed/thrust, connecting electrical and mechanical dynamics.

For current reproducible flight results, the adapter still uses declared first-order thrust lag and normalized linear thrust, while Gazebo integrates nonlinear rigid-body/contact dynamics. The richer equations above define the model to identify; no arbitrary electrical coefficients were invented or silently substituted into an already-recorded flight.

## 4.5 Why a single transfer function cannot represent all of this

Transfer functions describe linear time-invariant input/output models with specified initial conditions. Our exact equations contain $\Omega^2$, $T\cos\phi\cos\theta$, and $pq$ products. In general,

$$\mathcal{L}\{\Omega(t)^2\} \neq \left(\mathcal{L}\{\Omega(t)\}\right)^2. \tag{4.12}$$

We use the nonlinear equations for prediction and simulation, and a local Jacobian model for transfer-function/gain design. Multiple operating points or nonlinear control methods can extend this design later; pretending the full nonlinear aircraft has one global scalar transfer function would be incorrect.

# Chapter 5

# Equilibrium, local models and the six controllers

## 5.1 Keep the full model as the prediction model

Collect the nonlinear mechanical, rotor and electrical states:

$$\boldsymbol{x} = (\boldsymbol{r}^T, \boldsymbol{v}^T, q^T, \boldsymbol{\omega}^T, \Omega_1, \ldots, \Omega_4, I_1, \ldots, I_4, V_p, z)^T. \tag{5.1}$$

The preceding equations define $\dot{\boldsymbol{x}} = f(\boldsymbol{x}, \boldsymbol{u}, \boldsymbol{\theta})$, where $\boldsymbol{\theta}$ contains measured or assumed physical parameters. The unit quaternion has one constraint. A numerical simulation integrates these equations; it need not replace trigonometric functions with their small-angle forms.

For controller calculation, select a level hover operating point:

$$\boldsymbol{v}_0 = 0, \qquad \boldsymbol{\omega}_0 = 0, \qquad R_0 = I, \tag{5.2}$$

$$\sum_i k_T \Omega_{i0}^2 = mg, \qquad k_\tau I_{i0} = Q_i(\Omega_{i0}) + B_r \Omega_{i0}. \tag{5.3}$$

For symmetric rotors, $\Omega_{i0} = \sqrt{mg/(4k_T)}$. Without identified $k_T$, this is a relationship, not an available numeric RPM.

## 5.2 Where a local linear model comes from

Write $\boldsymbol{x} = \boldsymbol{x}_0 + \delta\boldsymbol{x}$ and $\boldsymbol{u} = \boldsymbol{u}_0 + \delta\boldsymbol{u}$. A first-order Taylor expansion at equilibrium gives

$$f(\boldsymbol{x}_0 + \delta\boldsymbol{x}, \boldsymbol{u}_0 + \delta\boldsymbol{u}) = f(\boldsymbol{x}_0, \boldsymbol{u}_0) + A\delta\boldsymbol{x} + B\delta\boldsymbol{u} + \text{higher-order terms}, \tag{5.4}$$

$$A = \left.\frac{\partial f}{\partial \boldsymbol{x}}\right|_0, \qquad B = \left.\frac{\partial f}{\partial \boldsymbol{u}}\right|_0. \tag{5.5}$$

At equilibrium $f(\boldsymbol{x}_0, \boldsymbol{u}_0) = 0$. Neglecting only the remainder gives the *local* perturbation model. This step is explicitly labelled and does not delete the original nonlinear equations.

For example, speed-squared thrust expands exactly as

$$T(\Omega_0 + \delta\Omega) = k_T\Omega_0^2 + 2k_T\Omega_0\delta\Omega + k_T(\delta\Omega)^2, \tag{5.6}$$

$$\delta T \approx 2k_T\Omega_0\delta\Omega. \tag{5.7}$$

The local slope $2k_T\Omega_0$ changes with operating point. For a measured command curve $T = f_T(u)$, $\delta T \approx f_T'(u_0)\delta u$. A nonlinear thrust curve can therefore be used for simulation while its measured hover slope supplies local controller gain.

## 5.3 The currently reproducible baseline

| Parameter | Assumed baseline |
|---|---|
| $m$ | 1.2 kg |
| $(J_x, J_y, J_z)$ | (.012,.012,.022) kg m$^2$ |
| $D$, $a = D/(2\sqrt{2})$ | .45 m, .159099 m |
| $T_{\max}$ | 6 N per rotor in the normalized linear-thrust surrogate |
| $\tau_m$ | .04 s thrust lag |
| $d$ | .12 N s/m |
| $\Delta t$ | .002 s virtual control interval |

For the surrogate $T_i = T_{\max} u_i^a$,

$$c_0 = \frac{mg}{4T_{\max}} = .4903325. \tag{5.8}$$

The other model is the provisional .740 kg component URDF, with inertia

$$(J_x, J_y, J_z) = (.00885852, .00906322, .01718682)\,\mathrm{kg\,m}^2. \tag{5.9}$$

Its masses are not verified and it is not the physics body of the recorded 1.2 kg flight. Never combine one model's mass with another model's inertia accidentally.

## 5.4 Derive each local transfer function

For a small roll perturbation, with other rates zero,

$$J_x \dot{p} = 4aT_{\max} u_\phi^a, \tag{5.10}$$

$$\tau_m \dot{u}_\phi^a + u_\phi^a = u_\phi. \tag{5.11}$$

In radian units define $b_x = 4aT_{\max}/J_x$. Under zero perturbation initial conditions, $\mathcal{L}\{\dot{p}\} = sP(s)$, so

$$sP(s) = b_x U_\phi^a(s), \tag{5.12}$$

$$(\tau_m s + 1) U_\phi^a(s) = U_\phi(s), \tag{5.13}$$

$$\frac{P(s)}{U_\phi(s)} = \frac{b_x}{s(\tau_m s + 1)}. \tag{5.14}$$

Integrating rate gives a further factor $1/s$ in the local angle plant:

$$G_\phi(s) = \frac{b_x}{s^2(\tau_m s + 1)}. \tag{5.15}$$

Pitch substitutes $J_y$. Yaw rate substitutes $b_z = 4\kappa T_{\max}/J_z$ and has no outer yaw-angle controller.

The present C rate inputs are degrees/s. Multiply the radian effectiveness by $180/\pi$ before using those C gain units:

$$b_{\phi,\mathrm{deg}} = 18231.4054, \qquad b_{\psi,\mathrm{deg}} = 937.5673. \tag{5.16}$$

For height, subtract hover equilibrium and set $\lambda = d/m$:

$$\delta\ddot{h} + \lambda\delta\dot{h} = b_h \delta c^a, \qquad b_h = \frac{4T_{\max}}{m}, \tag{5.17}$$

$$G_h(s) = \frac{b_h}{s(s+\lambda)(\tau_m s + 1)}. \tag{5.18}$$

The numeric baseline is $G_h(s) = 20/[s(s + .1)(.04s + 1)]$.

## 5.5 Why do not all controllers use nonzero P, I and D?

The letters name three available mathematical terms:

$$e(t) = r(t) - y(t), \tag{5.19}$$
$$u_P(t) = K_p e(t), \tag{5.20}$$
$$u_I(t) = K_i \int_0^t e(\xi)\,\mathrm{d}\xi, \tag{5.21}$$
$$u_D(t) = -K_d \dot{y}(t). \tag{5.22}$$

The last form differentiates measurement to avoid target-derivative kick. Choosing $K_i = 0$ or $K_d = 0$ is a design choice, not an incomplete calculation.

### 5.5.1 P changes the immediate response

Larger error gives larger correction. For an angle error of 3 degrees and outer $K_a = 3\,\mathrm{s}^{-1}$, the desired rate is 9 degrees/s. P-only angle control can work with an inner rate PI that rejects constant torque disturbances.

### 5.5.2 I adds a state and removes certain steady errors

For a constant rate disturbance, the inner rate integral can grow until its output balances the disturbance torque. But each integrator adds dynamic order and can accumulate unavailable demand during saturation. An additional outer angle integral is not automatically beneficial; sensor bias is not made physically correct merely by integrating its mistaken measurement.

### 5.5.3 D amplifies high-frequency measurement variation

If independent measurement noise has variance $\sigma_y^2$,

$$\dot{y}_k \approx (y_k - y_{k-1})/\Delta t, \tag{5.23}$$
$$\mathrm{Var}(\dot{n}_k) = \frac{2\sigma_y^2}{\Delta t^2}. \tag{5.24}$$

At .2 degrees/s gyro sigma and .002 s spacing, the unfiltered rate derivative noise sigma would be about 141.4 degrees/s$^2$. Real filtering/correlation changes that number, but not the need to account for noise. A nonzero rate D term therefore needs bandwidth/noise design, not merely a wish to use all letters.

### 5.5.4 Why altitude does use D

The altitude controller has a useful filtered velocity estimate. It forms

$$u_h = K_p(h_{\mathrm{ref}} - \hat{h}) + K_i I_h + K_d(v_{\mathrm{cmd}} - \hat{v}). \tag{5.25}$$

Since $\dot{h}_{\mathrm{ref}} \approx v_{\mathrm{cmd}}$ away from its limiter, the last term is a derivative of height error using an estimated state rather than a noisy finite difference. There is one altitude PID, not a seventh separate velocity PID.

## 5.6 Calculate the initial six gains

For a rate plant first approximated by $b/s$, use PI:

$$C(s) = \frac{K_p s + K_i}{s}, \tag{5.26}$$

$$1 + C(s)G(s) = 0, \tag{5.27}$$

$$s^2 + bK_p s + bK_i = 0. \tag{5.28}$$

Choose $s^2 + 2\zeta\omega_n s + \omega_n^2$ and equate coefficients:

$$K_p = \frac{2\zeta\omega_n}{b}, \qquad K_i = \frac{\omega_n^2}{b}. \tag{5.29}$$

For the current candidate, $\zeta = .9$, roll/pitch $\omega_n = 8.4$ rad/s, yaw $\omega_n = 2.8$ rad/s, and outer angle $K_a = 3$. These bandwidths are chosen response objectives, not motor measurements.

For the initial altitude double-integrator design,

$$G(s) = b_h/s^2, \tag{5.30}$$

$$1 + (K_p + K_i/s + K_d s)\, G(s) = 0, \tag{5.31}$$

$$s^3 + b_h K_d s^2 + b_h K_p s + b_h K_i = 0. \tag{5.32}$$

Expand desired $(s + p_h)(s^2 + 2\zeta_h\omega_h s + \omega_h^2)$:

$$s^3 + (2\zeta_h\omega_h + p_h)s^2 + (\omega_h^2 + 2\zeta_h\omega_h p_h)s + p_h\omega_h^2. \tag{5.33}$$

Coefficient matching gives

$$K_d = \frac{2\zeta_h\omega_h + p_h}{b_h}, \quad K_p = \frac{\omega_h^2 + 2\zeta_h\omega_h p_h}{b_h}, \quad K_i = \frac{p_h\omega_h^2}{b_h}. \tag{5.34}$$

Use $\omega_h = 1.05$, $p_h = .56$, $\zeta_h = .9$. The resulting degree-based C gains:

| Loop | $K_p$ | $K_i$ | $K_d$ |
|---|---|---|---|
| Roll angle | 3 | 0 | 0 |
| Roll rate | .0008293381 | .0038702447 | 0 |
| Pitch angle | 3 | 0 | 0 |
| Pitch rate | .0008293381 | .0038702447 | 0 |
| Yaw rate | .0053756141 | .0083620664 | 0 |
| Altitude | .108045 | .03087 | .1225 |

## 5.7 Now restore lag, drag and the complete cascade

For a rate PI with lag, the characteristic is

$$\tau_m s^3 + s^2 + bK_p s + bK_i. \tag{5.35}$$

Positive gains are not enough: the cubic Routh condition also requires $K_p > \tau_m K_i$. The baseline roll/pitch polynomial is $.04s^3 + s^2 + 15.12s + 70.56$, with poles $-7$ and $-9 \pm 13.0767j$.

The outer angle loop surrounds the closed inner rate loop plus angle integration:

$$T_p(s) = \frac{b(K_p s + K_i)}{\tau_m s^3 + s^2 + bK_p s + bK_i}, \tag{5.36}$$

$$1 + \frac{K_a T_p(s)}{s} = 0. \tag{5.37}$$

Multiplying out gives the complete cascade characteristic

$$\tau_m s^4 + s^3 + bK_p s^2 + b(K_i + K_a K_p)s + K_a b K_i. \tag{5.38}$$

For altitude with drag and lag:

$$\tau_m s^4 + (1 + \tau_m \lambda)s^3 + (\lambda + b_h K_d)s^2 + b_h K_p s + b_h K_i. \tag{5.39}$$

These quartics retain more of the local dynamics than the initial simplified coefficient-matching models.

For $a_4 s^4 + a_3 s^3 + a_2 s^2 + a_1 s + a_0$, the Routh first-column construction is

$$b_1 = \frac{a_3 a_2 - a_4 a_1}{a_3}, \tag{5.40}$$

$$c_1 = \frac{b_1 a_1 - a_3 a_0}{b_1}. \tag{5.41}$$

With all $a_i > 0$, positivity reduces to

$$\Delta_2 = a_3 a_2 - a_4 a_1 > 0, \tag{5.42}$$

$$\Delta_3 = a_3 a_2 a_1 - a_4 a_1^2 - a_3^2 a_0 > 0. \tag{5.43}$$

For the baseline angle cascade these are 10.4832 and 1003.532544. For altitude they are 2.473764 and 4.7232075492. Both local continuous models satisfy the conditions; this does not prove a delayed/saturated nonlinear flight.

### 5.7.1 A fuller altitude coefficient match

Choose the fourth-order desired polynomial

$$\tau_m(s + \rho)(s + p_h)(s^2 + 2\zeta_h \omega_h s + \omega_h^2). \tag{5.44}$$

The cubic coefficient fixes $\rho = 1/\tau_m + \lambda - p_h - 2\zeta_h \omega_h$. Equate the other coefficients:

$$K_d = \frac{\tau_m[\omega_h^2 + 2\zeta_h \omega_h p_h + \rho(p_h + 2\zeta_h \omega_h)] - \lambda}{b_h}, \tag{5.45}$$

$$K_p = \frac{\tau_m[p_h \omega_h^2 + \rho(\omega_h^2 + 2\zeta_h \omega_h p_h)]}{b_h}, \tag{5.46}$$

$$K_i = \frac{\tau_m \rho p_h \omega_h^2}{b_h}. \tag{5.47}$$

This gives the uninstalled analytic candidate $(.09912357, .02796822, .1103068)$ for $(K_p, K_i, K_d)$. It has selected local poles $-22.65$, $-.56$, $-.945 \pm .457684j$. It has not replaced the recorded-flight gains or been verified in a new flight.

## 5.8 Sampling, limits and nonlinear assessment

For a local continuous state model, zero-order-hold discretization is

$$A_d = e^{A\Delta t}, \qquad B_d = \int_0^{\Delta t} e^{A\xi} B \, \mathrm{d}\xi. \tag{5.48}$$

Augment the discrete plant with integral, estimator and delay states. Local sampled stability requires all closed-loop eigenvalues to satisfy $|z_i| < 1$. Continuous poles alone do not prove this implementation property.

Use the nonlinear model to examine different initial attitudes, wind, changing voltage, sensor errors, motor limits and parameter ranges. This is validation of the design model, not a reason to linearize away every difficult term. Saturation, ground contact and the battery disarm event are hybrid/non-smooth events and must be represented explicitly.

# Chapter 6

# Kalman estimation: uncertainty changes the gains

## 6.1 From noisy sensors to states

The nonlinear sensor model for a body accelerometer is

$$\boldsymbol{a}_m = R^T(\ddot{\boldsymbol{r}} - \boldsymbol{g}_W) + \boldsymbol{b}_a + \boldsymbol{n}_a, \qquad \boldsymbol{g}_W = (0, 0, -g)^T. \tag{6.1}$$

At rest and level, it reads approximately $(0, 0, g)^T$; in ideal free fall $\ddot{\boldsymbol{r}} = \boldsymbol{g}_W$ and it reads approximately zero. An accelerometer measures specific force, not a direct world acceleration vector. The gyro model is $\boldsymbol{g}_m = \boldsymbol{\omega} + \boldsymbol{b}_g + \boldsymbol{n}_g$.

Gravity-based tilt observations are

$$\phi_a = \operatorname{atan2}(a_y, a_z), \qquad \theta_a = \operatorname{atan2}(-a_x, \sqrt{a_y^2 + a_z^2}). \tag{6.2}$$

During translation these observations contain nongravitational acceleration. Using a stationary noise variance does not remove that modelling error.

## 6.2 The current two-state angle/bias filter

For one independent angular axis:

$$\boldsymbol{x} = (\alpha, b_g)^T, \qquad F = \begin{bmatrix} 1 & -\Delta t \\ 0 & 1 \end{bmatrix}, \qquad H = [1, 0], \tag{6.3}$$

$$\alpha^- = \alpha + \Delta t(g_m - b_g), \qquad b_g^- = b_g. \tag{6.4}$$

Although sometimes called a one-dimensional angle filter, it estimates two states per axis. Let $P = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$. Then

$$FPF^T = \begin{bmatrix} a - \Delta t(b + c) + \Delta t^2 d & b - \Delta t d \\ c - \Delta t d & d \end{bmatrix}. \tag{6.5}$$

The current noise model adds

$$Q = \begin{bmatrix} \sigma_g^2 \Delta t^2 + q_b \Delta t^3/3 & -q_b \Delta t^2/2 \\ -q_b \Delta t^2/2 & q_b \Delta t \end{bmatrix}. \tag{6.6}$$

Here $\sigma_g^2$ is rate variance per sample and $q_b$ is a continuous bias-random-walk intensity. They do not have interchangeable units.

## 6.3 Derive the correction quantities

$$P^- = FP^+F^T + Q, \tag{6.7}$$
$$\nu = z - H\boldsymbol{x}^-, \tag{6.8}$$
$$S = HP^-H^T + R, \tag{6.9}$$
$$K = P^-H^TS^{-1}, \tag{6.10}$$
$$\boldsymbol{x}^+ = \boldsymbol{x}^- + K\nu. \tag{6.11}$$

For $H = [1, 0]$, $S = P^-_{00}+R$ and $K = (P^-_{00}, P^-_{10})^T/S$. A scalar observation can correct bias through cross covariance even though it does not directly measure bias. The covariance uses Joseph form

$$P^+ = (I - KH)P^-(I - KH)^T + KRK^T. \tag{6.12}$$

With the existing assumed noise values and .002 s updates, limiting gains are approximately $K_\alpha = .000966191$, $K_b = -.000223499$. They are conditional on the model; startup gains differ and they are not universal sensor constants.

## 6.4 Height and vertical velocity

For $\boldsymbol{x} = (h, v_z)^T$, external upward acceleration and interval $\Delta t$:

$$F = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix}, \qquad G = \begin{bmatrix} \Delta t^2/2 \\ \Delta t \end{bmatrix}, \tag{6.13}$$
$$\boldsymbol{x}^- = F\boldsymbol{x}^+ + Ga_{\text{up}}, \qquad Q = G\sigma_a^2G^T. \tag{6.14}$$

With $\sigma_a = 1.5$ m/s$^2$, $\Delta t = .002$ s:

$$Q = \begin{bmatrix} 9\,10^{-12} & 9\,10^{-9} \\ 9\,10^{-9} & 9\,10^{-6} \end{bmatrix}. \tag{6.15}$$

Predict at 500 Hz, correct only on independent fresh observations, typically 50 Hz for the assumed barometer. With $R_{\text{baro}} = .0025$ m$^2$, periodic limiting gains at correction time are approximately $(.0834306, .181649)^T$. Applying these as constant gains every 2 ms would be a different filter.

An observed height from a tilted downward range sensor also needs its beam geometry, sensor lever arm and terrain assumptions. A one-ray range is not automatically aircraft vertical height over arbitrary obstacles. The current simulation's floor ray is declared analytical, not a native physical sensor.

## 6.5 A fuller estimator is possible, but must be identified

For a nonlinear model $\dot{x} = f(x, u)$ and observations $y = h(x)$, an EKF uses Jacobians of $f, h$ along the evolving estimate. A quaternion error-state filter can include position, velocity, attitude and IMU biases. It preserves more of the nonlinear observation geometry than independent tilt filters. This is a future estimator option, not an implementation claim. Unknown noise, unobservable states and poor camera/pose information do not disappear merely because a larger filter is used.

# Chapter 7

# Standard deviation is not the same as knowing the true value

## 7.1 A fixed mass and fluctuating measurements

Suppose the completed drone has fixed mass $m$. A scale gives

$$y_k = m + b + \epsilon_k. \tag{7.1}$$

$b$ is an unknown calibration offset and $\epsilon_k$ represents changing measurement effects. Repeating readings estimates dispersion. It does not make the aircraft's mass fluctuate or reveal the offset by itself.

For $N$ readings:

$$\bar{y} = \frac{1}{N}\sum_{k=1}^{N} y_k, \tag{7.2}$$

$$s^2 = \frac{1}{N-1}\sum_{k=1}^{N}(y_k - \bar{y})^2, \qquad s = \sqrt{s^2}, \tag{7.3}$$

$$u_A(\bar{y}) = \frac{s}{\sqrt{N}} \quad \text{if samples are independent.} \tag{7.4}$$

$s$ describes individual-reading dispersion; $u_A$ describes uncertainty of their average from repeatability. Repeating a biased scale 10000 times reduces $u_A$ but does not remove $b$. A characterized reference is needed to assess bias.

## 7.2 Type A and Type B evaluation

Type A means evaluated statistically from data. Type B means evaluated using other information such as a calibration certificate, resolution or a justified physical bound. These describe evaluation methods, not merely random versus systematic physical effects.

For certificate expanded uncertainty $U$ and coverage factor $k$:

$$u_{\text{certificate}} = U/k. \tag{7.5}$$

For rounding interval width $q$, if error is reasonably uniform on $[-q/2, q/2]$:

$$u^2_{\text{resolution}} = \frac{1}{q}\int_{-q/2}^{q/2} x^2 \,\mathrm{d}x = \frac{q^2}{12}, \tag{7.6}$$

$$u_{\text{resolution}} = q/\sqrt{12}. \tag{7.7}$$

A justified uniform bound $[-a, a]$ gives $u = a/\sqrt{3}$. A triangular distribution on those bounds gives $u = a/\sqrt{6}$. The distribution is part of the assumption, not something every tolerance label automatically specifies.

## 7.3 Combine uncertainty contributions

For independent additive contributions:

$$u_c^2 = u_A^2 + u_{\text{certificate}}^2 + u_{\text{resolution}}^2 + \cdots . \tag{7.8}$$

If repeated data already capture quantization variation, account for that overlap rather than counting the same source twice. For correlated quantities keep their covariance. In general, for $\boldsymbol{y} = f(\boldsymbol{\theta})$:

$$J_f = \frac{\partial f}{\partial \boldsymbol{\theta}}, \tag{7.9}$$

$$\Sigma_y \approx J_f \Sigma_\theta J_f^T. \tag{7.10}$$

For one scalar:

$$u_y^2 \approx \sum_i \left(\frac{\partial f}{\partial \theta_i}\right)^2 u_i^2 + 2\sum_{i<j} \frac{\partial f}{\partial \theta_i}\frac{\partial f}{\partial \theta_j}\operatorname{Cov}(\theta_i, \theta_j). \tag{7.11}$$

This is a first-order approximation. Strong nonlinearity may require propagation of a justified joint distribution, not invented independent Gaussian inputs.

### 7.3.1 Worked example: not aircraft measurements

Invented teaching readings in grams are $1200, 1201, 1199, 1200, 1200$. They give $\bar{y} = 1200$ g, $s = .7071$ g and $u_A = .3162$ g. If separate residual scale-calibration standard uncertainty is 1 g and a separately justified resolution contribution is .2887 g:

$$u_c = \sqrt{.3162^2 + 1^2 + .2887^2} \approx 1.0878\,\text{g}. \tag{7.12}$$

An illustrative $k = 2$ expanded uncertainty is 2.18 g. Its coverage interpretation needs assumptions about distributions and degrees of freedom. This is not a measured confidence interval for IntelQcopter.

## 7.4 Correlated samples and changing conditions

For stationary normalized lag correlations $\rho_j$, an approximate effective sample count is

$$N_{\text{eff}} \approx \frac{N}{1 + 2\sum_{j=1}^{J}(1 - j/N)\rho_j}. \tag{7.13}$$

Then $s/\sqrt{N_{\text{eff}}}$ can be more appropriate. Lag window $J$ and stationarity need judgment. Temperature drift, mounting stress and motion can invalidate one stationary-noise calculation.

In practice the reference also has uncertainty. Estimate a correction and retain its uncertainty:

$$y_{\text{corrected}} = y_{\text{instrument}} - \hat{b}. \tag{7.14}$$

The corrected result's budget must include $u(\hat{b})$. For a fitted gain, retain coefficient covariance and residual behavior. Independent validation points check predictions outside the points used to fit. A small fitting residual does not establish infinitely precise truth.

# Chapter 8

# Practical identification of the aircraft

## 8.1 Records and measurement priority

Blank CSV templates are under `measurements/templates`. They are empty, not synthetic evidence. Record units, timestamps, instrument identity, calibration status, sensor/ESC settings, mounting, temperature and battery voltage. Preserve raw readings before filtering.

1. Completed takeoff mass and centre of mass.
2. Rotor locations and assembled inertia.
3. Static thrust/current/RPM versus command and battery voltage.
4. Command-to-thrust/RPM delay and transients near hover.
5. Sensor noise, bias, scale and temperature effects.
6. Electrical calibration, capacity, camera geometry and timing/communication.

These procedures can be prepared without selecting a Pi/Jetson board now.

## 8.2 Mass and centre of mass

Weigh the full flight configuration, including ESCs, wiring, fasteners and payload. Earlier neglect of small components does not justify excluding them from final measurements. Check zero and an independently characterized reference mass near the working range. Weigh, remove and replace the aircraft repeatedly to include placement variation. Repeat after battery/payload changes.

For component COM positions $\boldsymbol{r}_i$:

$$m = \sum_i m_i, \qquad \boldsymbol{r}_C = \frac{\sum_i m_i \boldsymbol{r}_i}{m}. \tag{8.1}$$

For a stationary two-support balance at known positions $x_1, x_2$:

$$R_1 + R_2 = mg, \tag{8.2}$$

$$R_1 x_1 + R_2 x_2 = m g x_C, \tag{8.3}$$

$$x_C = \frac{R_1 x_1 + R_2 x_2}{R_1 + R_2}. \tag{8.4}$$

Repeat in the other direction. Unknown support moments or contact constraints invalidate this simple force/moment balance.

## 8.3 Inertia from components

For a box with dimensions $l_x, l_y, l_z$ about its own COM:

$$J_{\text{box}} = \frac{m}{12}\,\text{diag}(l_y^2 + l_z^2, l_x^2 + l_z^2, l_x^2 + l_y^2). \tag{8.5}$$

Rotate each tensor into common axes and shift it to the aircraft COM:

$$J = \sum_i \left[R_i J_i R_i^T + m_i((\boldsymbol{d}_i^T \boldsymbol{d}_i)I - \boldsymbol{d}_i \boldsymbol{d}_i^T)\right], \quad \boldsymbol{d}_i = \boldsymbol{r}_i - \boldsymbol{r}_C. \tag{8.6}$$

This is the matrix parallel-axis theorem. Approximate geometry still gives an approximate tensor even when the algebra is exact.

## 8.4 Inertia from a bifilar pendulum

Two equal parallel strings have length $L$, half separation $a$. For small torsion angle $\theta$ about the appropriate vertical COM axis, geometry gives

$$\Delta h \approx \frac{a^2\theta^2}{2L}. \tag{8.7}$$

Potential energy is $U = mg\Delta h$, hence

$$\tau_{\text{restore}} = -\frac{\partial U}{\partial \theta} = -\frac{mga^2}{L}\theta, \tag{8.8}$$

$$J\ddot{\theta} + \frac{mga^2}{L}\theta = 0, \tag{8.9}$$

$$\omega_n = \sqrt{\frac{mga^2}{LJ}}, \qquad \mathcal{T} = \frac{2\pi}{\omega_n}, \tag{8.10}$$

$$J = \frac{mga^2\mathcal{T}^2}{4\pi^2 L}. \tag{8.11}$$

Measure $\mathcal{T} = t_{\text{elapsed}}/N_{\text{cycles}}$ over several cycles; repeat at several small amplitudes to check the approximation. Reorient the unpowered assembly with a correctly aligned fixture for other axes. Coupled swinging is not a valid torsion record. Remove props for these non-powered mechanical tests.

If a fixture rotates too, subtract its independently characterized inertia about the same axis, not its period: $J_{\text{aircraft}} = J_{\text{total}} - J_{\text{fixture}}$. For independent input uncertainties:

$$\left(\frac{u_J}{J}\right)^2 \approx \left(\frac{u_m}{m}\right)^2 + 4\left(\frac{u_a}{a}\right)^2 + 4\left(\frac{u_\mathcal{T}}{\mathcal{T}}\right)^2 + \left(\frac{u_L}{L}\right)^2. \tag{8.12}$$

COM/axis alignment, fixture flexibility and non-small motion add model errors.

## 8.5 Static thrust, current and RPM

Use the actual motor, ESC and propeller type on a secured force fixture with a guard/exclusion arrangement. Never hold a running drone/motor by hand. Check airflow clearance empirically: increasing clearance should stop changing the steady result appreciably.

1. Calibrate the entire force path using known forces; verify sign, lever ratio, tare and preload.

2. Record zero command, terminal voltage and temperature.
3. At several commands, wait for a steady plateau; record force, DC current, voltage and RPM where available.
4. Repeat increasing/decreasing commands to expose hysteresis, heating and sag.
5. Repeat near the expected hover thrust and at relevant pack voltages, within verified component/fixture limits.

A scale's displayed gram is not a newton: 1 gram mass-equivalent corresponds to .00980665 N. Upward thrust can reduce or increase a reading depending on fixture geometry. Use the calibrated signed change and mechanical ratio.

Fit $T(u, V)$ rather than imposing linearity. For a local quadratic:

$$T(u) = \beta_0 + \beta_1 u + \beta_2 u^2, \tag{8.13}$$

$$T'(u_0) = \beta_1 + 2\beta_2 u_0, \tag{8.14}$$

$$4T(u_0, V_0) = mg. \tag{8.15}$$

Use repeated points for scatter and independent points for validation. With RPM measurements, $\Omega = 2\pi\,\mathrm{RPM}/60$ permits a fit of $T = k_T\Omega^2$; examine residuals and range of applicability.

## 8.6 Response delay, time constant and model order

A steady scale can have unknown internal filtering and insufficient bandwidth for a 40 ms transient. Use a timestamped force/RPM chain with adequate mechanical, analog and digital bandwidth. A 500–1000 Hz logger gives many samples across 40 ms, but does not itself prove a fast force sensor.

For an accepted first-order model with delay $t_d$:

$$T(t) = T_0 + (T_\infty - T_0)\left(1 - e^{-(t-t_d)/\tau_m}\right), \quad t \geq t_d. \tag{8.16}$$

At $t_d + \tau_m$:

$$\frac{T - T_0}{T_\infty - T_0} = 1 - e^{-1} \approx .63212. \tag{8.17}$$

Use this for an initial estimate, then fit the complete transient. Capture repeated small up/down steps near hover at several voltages with actual command timestamps. If residuals reveal oscillation, two timescales or asymmetric rise/fall, retain a higher-order/nonlinear model. Separate ESC/transport delay from mechanical lag.

## 8.7 Yaw torque

A torque fixture can measure reaction force $F$ at a known tangential lever $\ell$:

$$Q = \ell F, \tag{8.18}$$

$$u_Q^2 \approx F^2 u_\ell^2 + \ell^2 u_F^2 + 2\ell F\,\mathrm{Cov}(\ell, F). \tag{8.19}$$

Record simultaneous thrust/RPM and fit $Q(\Omega)$ or $Q(T)$. The current $\kappa = .015$ m is assumed, not measured. Isolate reaction torque from off-centre thrust, support friction and other fixture moments.

## 8.8 IMU bias, noise, scale and temperature

Log fresh timestamps, axes, physical units, full-scale/DLPF/sample settings and temperature on a fixed support with motors unpowered. Observe warm-up trends; do not invent a universal fixed warm-up time.

1. Stationary gyro mean estimates zero-rate offset, not gyro scale factor.
2. Residuals around that mean estimate sample noise under those settings.
3. Six accelerometer faces estimate diagonal gain/offset.
4. Known reference angle/rate motion is needed for gyro scale/cross-axis error.
5. Repeat at stable temperatures; fit temperature behavior only if data support it.

For $b_i(T) = a_i + c_i(T - T_{\text{ref}})$, retain coefficient covariance and residuals. Do not assign a slope merely because bias is temperature dependent.

One timescale statistic is Allan deviation. For successive non-overlapping cluster means of duration $\tau$:

$$\sigma_A^2(\tau) = \frac{1}{2(M-1)} \sum_{j=1}^{M-1} (\bar{y}_{j+1}(\tau) - \bar{y}_j(\tau))^2. \tag{8.20}$$

Its dependence on $\tau$ helps expose white noise and drift, but needs long enough records and does not automatically identify every process coefficient.

## 8.9 Height, range and camera measurements

**Barometer:** log compensated pressure at fixed height and temperature, then validate several independently measured height differences. Distinguish slow reference/weather drift from white noise.

**Lidar:** use independently measured target distances, multiple surfaces, angles and lighting; preserve invalid returns. Measure beam direction and sensor-to-body lever arm. Slant distance is not vertical height without geometry.

**Camera:** record identity/resolution/focus and measured target dimensions. Capture varied target poses, fit intrinsics/distortion, validate on independent poses and measure camera-to-body extrinsics. Reprojection error is not automatically metric obstacle-range error. Arbitrary unknown object scale is not observable from one image alone.

## 8.10 Voltage, current, capacity and timing

**Voltage:** compare the divider/ADC to a characterized meter at several voltages. Fit gain/offset and keep reference/fit uncertainty. **Current:** compare with a suitable DC reference over the intended range; measure zero, sign, gain and temperature effects. A voltage divider alone does not provide current.

**Capacity:** use an appropriate controlled charger and discharger, and integrate

$$C_{\text{measured}} = \int I \, \mathrm{d}t/3600 \tag{8.21}$$

under stated temperature, current and cutoff conditions. Do not measure usable capacity by flying until the motors stop. Pack labels are not a complete capacity/available-power model. SOC is an uncertain estimate: crossing 20 percent does not prove exactly 20 percent of physical usable capacity remains. Starting charge, effective capacity, current calibration and temperature affect that estimate.

**Timing:** record acquisition, reception, computation and actuation timestamps. A timing GPIO plus suitable instrument can measure local execution. One-way distributed latency needs synchronized clocks or another method that separates latency from clock offset. Keep distributions and worst observed values.

## 8.11 How measurements improve gains

Replace the normalized surrogate $T_{\max}$ by measured hover slope $g_T = T'(u_0)$:

$$b_\phi = \frac{4ag_T}{J_x}, \qquad b_h = \frac{4g_T}{m}. \tag{8.22}$$

For fixed desired bandwidth, rate gains scale as $J_x/(ag_T)$:

$$\frac{\delta K_p}{K_p} \approx \frac{\delta J_x}{J_x} - \frac{\delta a}{a} - \frac{\delta g_T}{g_T}. \tag{8.23}$$

Propagate their joint covariance, including shared measurement sources. Sensor data inform observation covariance $R$ and bias/process assumptions for $Q$. A mass standard deviation cannot be copied into a sensor matrix with different physical units.

# Chapter 9

# Calibration storage and battery behavior

## 9.1 Persistent fits versus volatile reference data

The RP2350 build reserves two flash erase sectors, 8192 bytes total, outside the linked firmware region. Each record stores schema, generation, board identifier, sensor-profile version, validity mask, gyro count, IMU fits/uncertainties, magnetic offset/scale and voltage/current/lidar fits.

Explicit little-endian integer/binary32/binary64 encoding avoids struct padding. A CRC32 checks the data and a separate final page commits it. Ground barometer reference, live SOC and PID integrals are not persisted. Ground pressure must be measured again for the current session.

## 9.2 The journal write sequence

1. Find the newest complete record with matching CRC and commit page.
2. Prepare the next record in RAM, keeping the previous sector untouched.
3. Erase the other sector and program its data pages.
4. Program the commit page last; then reread the data.
5. Boot ignores incomplete/bad-CRC writes and selects the newest complete record.

This addresses ordinary interrupted writes if the other sector remains intact; it is not a physical guarantee for every brownout or flash failure. Hardware power-loss/endurance verification remains necessary. CRC is not authentication.

Board/profile mismatch or invalid fit ranges require recalibration. A sensor replacement on the same board must invalidate the fit/profile explicitly; board identity alone cannot identify every changed component. Clearing calibration writes a newer empty record so an older fit is not silently resurrected on normal reboot.

## 9.3 Verify the gyro after boot

Saved bias is loaded, then fresh stationary data verify it. For old/new bias fits and their estimated variances:

$$|\hat{b}_{\text{new}} - \hat{b}_{\text{old}}| > \max(.05,\ 5\sqrt{P_{\text{old}} + P_{\text{new}}}) \tag{9.1}$$

in degrees/s requests a refresh on any axis. Missing gyro fits also refresh. The current criterion is provisional engineering policy, not a measured temperature model. Unstable captures do not complete verification.

Only after verification and required fits exist can the sample helper mark an IMU sample calibrated. Missing accelerometer scale needs six-face calibration; level-only capture cannot recover all scale factors. The Pico's live sensor driver is still absent, so this code never invents capture data.

## 9.4 Programming conditions and commands

Flash programming conflicts with execute-in-place reads. The RAM write callback uses Pico SDK `flash_safe_execute`; the other core initializes participation in lockout. Writes are accepted only on core 0 while the caller is disarmed. Linker exclusion and a runtime overlap check protect the firmware area. Unchanged fits do not erase again; failed automatic saves retry no faster than every five seconds. After three automatic failures, retries stop until a manual S command or a newly installed fit resets the retry allowance.

| USB command | Meaning |
|---|---|
| C | Print loaded/missing calibration, gyro verification and unsaved status. |
| R | Persist an empty record and request recapture/recalculation. |
| S | Save valid measured fits, if any exist. |

These commands do not generate observations. Full-chip erase loses calibration. Configure actual physical flash size and board before flashing a custom PCB.

## 9.5 Requested SOC policy

For charge fraction $z$, the selected policy is

$$A(z) = \begin{cases} \text{normal}, & 100z > 35, \\ \text{warning}, & 20 < 100z \leq 35, \\ \text{latched disarm}, & 100z \leq 20. \end{cases} \tag{9.2}$$

The 35-percent warning flag is available for telemetry; current firmware prints transitions on USB when a monitor supplies data. Handheld LoRa notification is not yet connected. There is no live voltage/current acquisition, so the firmware does not pretend it knows real pack SOC.

At 20 percent the C mathematical demands are inhibited, regardless of height. The latch stays set if a later SOC estimate rebounds. An explicit reset while disarmed starts a new battery/session policy state. Critical external BMS/cell alerts also latch disarm; ordinary voltage/current warnings notify. There is no automatic battery landing request. Physical arming/ESC output remains disabled.

The simulation bridge accepts a manually supplied estimated SOC percentage (Float32) on this topic:

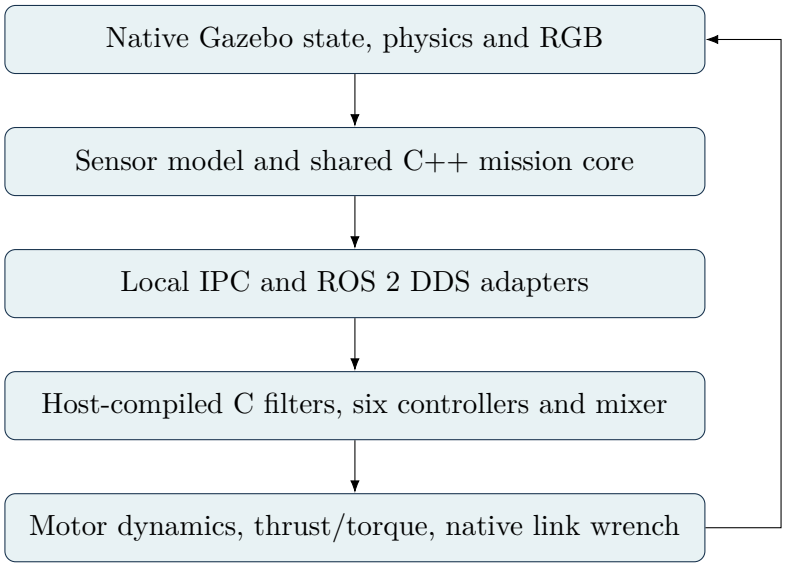
```
/qcopter/battery_soc_percent
```

It publishes UInt8 flags on `/qcopter/battery_policy`: bit 0 warning, bit 1 latched disarm. These reach the same host C instance that computes the flight demands. No new ROS flight has exercised this interface yet; it does not manufacture a battery discharge measurement.

# Chapter 10

# ROS, Gazebo and implementation boundaries

## 10.1 Current simulation feedback path




The host library runs production C math, not an RP2350 emulator. Virtual motor gating is separate from physical arming. The recorded flight uses assumed actuator dynamics, synthetic inertial/height observations, native camera pixels and simulated local odometry. Its downward range is an analytical floor ray. The full standard ROS C++ graph was built and checked separately, not silently substituted as the recorded flight path.

## 10.2 Future physical companion

ROS mission → USB/UART bridge → RP2350 parser → local 500 Hz control.

Physical framing, sequence/session, CRC, freshness, status/acknowledgement, clock handling and command authority remain work. ROS angle units are radians; the current C interface uses degrees. Convert at one documented boundary. A slower companion setpoint stream does not clock every inner PID iteration.

## 10.3 Build evidence and limits

The current simulator launcher is

```
powershell -ExecutionPolicy Bypass -File tools/run_native_garden.ps1
```

After editing the C source, rebuild host libraries using `tools/build_sitl.ps1`. The separate Pico build produces ELF/UF2. No physical board was flashed. Compilation does not prove flash durability, sensor acquisition, telemetry delivery or actual ESC inhibition. No new flight or implementation test suite was executed for this change.

# References and reproducibility

- Raspberry Pi Pico SDK flash APIs and execution safety: https://www.raspberrypi.com/documentation/pico-sdk/hardware.html. The installed `pico/flash.h` and linker/flash implementation were inspected.
- NIST uncertainty evaluation: Technical Note 1297, Type A evaluation, Type B evaluation.
- ROS axes/units: REP 103: standard units and coordinate conventions.
- TDK InvenSense MPU6000/MPU6050 specification: units, noise and temperature dependence; actual sensor configuration still needs characterization.
- Reproducible local model calculations: `tools/calculate_current_control_design.py` and the corresponding `artifacts/model-design/current/calculations.json`.
- Single-file runtime source: `flight_controller/flight_controller.c`.

Teaching numbers and scenario coefficients are labelled assumptions. No synthetic example is presented as measured aircraft data. Preserve raw measurements, reference uncertainty and source/configuration with each result.